Document number: P3111R3.
Date: 2025-01-13.
Authors: Gonzalo Brito Gadeschi, Simon Cooksey, Daniel Lustig.
Reply to: Gonzalo Brito Gadeschi <gonzalob _at_ nvidia.com (http://nvidia.com)>.
Audience: EWG, LEWG.

Table of Contents

Changelog

- **R3: pre Austria**:
  - Refactor atomic reduction sequence replacements into tables.
  - Provide alternative wording for atomic reduction sequence replacement using "as if" integer operations to provide same valid replacements as for integer reduction operations.
- **R2: post Wroclaw**: simplifies reduction sequence wording.
- **R1: post St. Louis '24**
  - *Overview of changes*: incorporate SG6 and SG1 feedback into the proposal, which resolved all open questions. Restructure exposition about what's being proposed accordingly.

- [SG6 Review]:
  - Forward it; does not need to look at it again.
  - Agree on "generalized" reductions being the default and not providing version without.
  - Do not use `GENERALIZED_SUM` to specify non-associative floating-point atomic reduction operations since it does not make sense for two values. Instead, attempt to add a `std::non_associative_add` operation that is exempted from the C standard clause in Annex F that makes re-ordering non-conforming. Recommended discussing this with CWG, which I did, and caused us to end up pursuing a different alternative (defining "reduction sequences") which is pursued in R1 instead.
- [SG1 Review]: Add wording for reduction sequences.
- [SG1 Review]: Add `compare_store` operation.
- [SG1 Review]: Update to handle `fenv` and `errno` for floating-point atomics.
- [SG1 Review]: Make "generalized" atomic floating-point reductions the default and do not provide fallback (beyond `fetch_<op>`).
- [SG1 Review]: Provide unsequenced support.
- [SG1 Review]: polls taken:

  1. Reduction operations are not a step, but infinite loops around reductions operations are not UB (practically implementations can assume reductions are finite)

SF	F	N	A	SA
3	9	1	0	0

  2. The single floating point reduction operations should allow "fast math" (allow tree reductions), you can get precise results using fetch_add/sub

SF	F	N	A	SA
5	5	1	1	0

  3. Any objection to approve the design of P3111R0: No objection.

- **R0: initial revision**.

Atomic Reduction Operations

Atomic Reduction Operations are atomic read-modify-write (RMW) operations (like `fetch_add`) that do not "fetch" the old value and are not reads for the purpose of building synchronization with acquire fences. This enables implementations to leverage hardware acceleration available in modern CPU and GPU architectures.

Furthermore, we propose to allow atomic memory operations that aren't reads in unsequenced execution, and to extend atomic arithmetic reduction operations for floating-point types with operations that assume floating-point arithmetic is associative.

Introduction

Concurrent algorithms performing atomic RMW operations that discard the old fetched value are very common in high-performance computing, e.g., finite-element matrix assembly, data analytics (e.g. building histograms), etc.

Consider the following parallel algorithm to build a histogram (full implementation (https://clang.godbolt.org/z/zscrhEPYj)):

```
// Example 0: Histogram.
span<unsigned> data;

array<atomic<unsigned>, N> buckets;
constexpr T bucket_sz = numeric_limits<T>::max() / (T)N;
unsigned nthreads = thread::hardware_concurrency();
for_each_n(execution::par_unseq, views::iota(0).begin(), nthreads,
 [&](int thread) {
  unsigned data_per_thread = data.size() / nthreads;
  T* data_thread = data.data() + data_per_thread * thread;
  for (auto e : span<T>(data_thread, data_per_thread))
    buckets[e / bucket_sz].fetch_add(1, memory_order_relaxed);
});
```

This program has two main issues:

- **Correctness** (undefined behavior): The program should have used the `par` execution policy to avoid undefined behavior since the atomic operation is not:
  - Potentially concurrent and therefore exhibits a data-race in unsequenced contexts ([intro.execution]).
  - Vectorization-safe [algorithms.parallel.defns#5] (https://eel.is/c++draft/algorithms.parallel.defns#5) since it is specificed to synchronize with other function invocations.
- **Performance**: Sophisticated compiler analysis required to optimize the above program for scalable hardware architectures with atomic reduction operations.

Atomic reduction operations address both shortcomings:

Before (compiler-explorer (https://clang.godbolt.org/z/xq8efq1WK))	**After**

`#include <algorithm>`
`#include <atomic>`
`#include <execution>`
`using namespace std;`
`using execution::par_unseq;`
 	
`int main() {`
`  size_t N = 10000;`
`  vector<int> v(N, 0);`
`  atomic<int> atom = 0;`
`  for_each_n(par_unseq,`
`  v.begin(), N,`
`  [&](auto& e) {`
`  // UB+SLOW:`
`  atom.fetch_add(e);`
`  });`
`  return atom.load();`
`}`
	`#include <algorithm>`
	`#include <atomic>`
	`#include <execution>`
	`using namespace std;`
	`using execution::par_unseq;`
	
	`int main() {`
	`  size_t N = 10000;`
	`  vector<int> v(N, 0);`
	`  atomic<int> atom = 0;`
	`  for_each_n(par_unseq,`
	`  v.begin(), N,`
	`  [&](auto& e) {`
	`  // OK+FAST`
	`  atom.reduce_add(e);`
	`  });`
	`  return atom.load();`
	`}`

This new operation can then be used in the Histogram Example (example 0), to replace the `fetch_add` with `reduce_add`.

Motivation

Hardware Exposure

The following ISAs provide Atomic Reduction Operation:

Architecture	Instructions
PTX	`red` (https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#parallel-synchronization-and-communication-instructions-red).
ARM	`LDADD RZ` (https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/LDADD--LDADDA--LDADDAL--LDADDL--Atomic-add-on-word-or-doubleword-in-memory-?lang=en), `STADD` (https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/STADD--STADDL--Atomic-add-on-word-or-doubleword-in-memory--without-return--an-alias-of-LDADD--LDADDA--LDADDAL--LDADDL-), `SWP RZ` (https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/SWP--SWPA--SWPAL--SWPL--Swap-word-or-doubleword-in-memory-?lang=en), `CAS RZ` (https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/CAS--CASA--CASAL--CASL--Compare-and-Swap-word-or-doubleword-in-memory-?lang=en).
x86-64	Remote Atomic Operations (RAO) (https://cdrdv2-public.intel.com/671368/architecture-instruction-set-extensions-programming-reference.pdf): AADD, AAND, AOR, AXOR.
RISC-V	None (note: AMOs are always loads and stores).
PP64LE	None.

Some of these instructions lack a destination operand (`red` (https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#parallel-synchronization-and-communication-instructions-red), `STADD` (https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/STADD--STADDL--Atomic-add-on-word-or-doubleword-in-memory--without-return--an-alias-of-LDADD--LDADDA--LDADDAL--LDADDL-), AADD). Others change semantics if the destination register used discards the result (Arm's `LDADD RZ` (https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/LDADD--LDADDA--LDADDAL--LDADDL--Atomic-add-on-word-or-doubleword-in-memory-?lang=en), `SWP RZ` (https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/SWP--SWPA--SWPAL--SWPL--Swap-word-or-doubleword-in-memory-?lang=en), `CAS RZ` (https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/CAS--CASA--CASAL--CASL--Compare-and-Swap-word-or-doubleword-in-memory-?lang=en)).

All ISAs provide the same sematics: these are not loads from the point-of-view of the Memory Model, and therefore do not participate in acquire sequences, but they do participate in release sequences:

- PTX Specification: `red` is "not a read operation (https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#operation-types)".
- Arm ARM: "where the destination register is WZR or XZR, are not regarded as doing a read for the purpose of a DMB LD barrier (https://developer.arm.com/documentation/ddi0487/latest)".
- x86-64: "since they do not load data from memory into the processor." (https://cdrdv2-public.intel.com/671368/architecture-instruction-set-extensions-programming-reference.pdf)

These architectures provide both "relaxed" and "release" orderings for the reductions (e.g. `red.relaxed` / `red.release`, `STADD` / `STADDL`).

Performance

On hardware architectures that implement these as far atomics, the exposed latency of Atomic Reduction Operations may be as low as half that of "`fetch_<key>`" operations.

Example: on an NVIDIA Hopper H100 GPU, replacing `atomic.fetch_add` with `atomic.reduce_add` on the Histogram Example (Example 0) improves throughput by 1.2x.

Furthermore, non-associative floating-point atomic operations, like `fetch_add`, are required to read the "latest value", which sequentializes their execution. In the following example, the outcome `x == a + (b + c)` is not allowed, because either the atomic operation of thread0 happens-before that of thread1, or vice-versa, and floating-point arithmetic is not associative:

```
// Litmus test 2:
atomic<float> x = a;
void thread0() { x.fetch_add(b, memory_order_relaxed); }
void thread1() { x.fetch_add(c, memory_order_relaxed); }
```

Allowing the `x == a + (b + c)` outcome enables implementations to perform a tree-reduction, which improves complexity from `O(N)` to `O(log(N))`, at negligible higher amount of non-determinism which is already inherent to the use of atomic operations:

```
// Litmus test 3:
atomic<float> x = a;
x.reduce_add(b, memory_order_relaxed);
x.reduce_add(c, memory_order_relaxed);
// Sound to merge these two operations into one:
// x.reduce_add(b + c);
```

On GPU architectures, performing an horizontal reduction for then issuing a single atomic operation per thread group, reduces the number of atomic operation issued by up to the size of the thread group.

Functionality

Currently, all atomic memory operations are vectorization-unsafe (https://eel.is/c++draft/algorithms.parallel.defns#5) and therefore not allowed in element access functions of parallel algorithms when the `unseq` or `par_unseq` execution policies are used (see [algorithms.parallel.exec.5] (https://eel.is/c++draft/algorithms.parallel#exec-5) and

[algorithms.parallel.exec.7] (https://eel.is/c++draft/algorithms.parallel#exec-7)). Atomic memory operations that "read" (e.g. `load`, `fetch_<key>`, `compare_exchange`, `exchange`, ...) enable building synchronization edges that block, which within `unseq` / `par_unseq` leads to dead-locks. N4070 (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4070.html) solved this by tightening the wording to disallow any synchronization API from being called from within `unseq` / `par_unseq`.

Allowing Atomic Writes and Atomic Reduction Operations in unsequenced execution increases the set of concurrent algorithms that can be implemented in the lowest-common denominator of hardware that C++ supports. In particular, many hardware architectures that can accelerate `unseq` / `par_unseq` but cannot accelerate `par` (e.g. most non-NVIDIA GPUs), provide acceleration for atomic reduction operations.

We propose to make lock-free atomic operations that are not reads vectorization safe to enable calling them from unsequenced execution. Atomic operations that read remain vectorization-unsafe and therefore UB:

Design

For each atomic `fetch_{OP}` in the `atomic<T>` and `atomic_ref<T>` class templates and their specializations, we introduce new `reduce_{OP}` member functions that return `void`:

```
template <class T>
struct atomic_ref {
  T    fetch_add (T v, memory_order o) const noexcept;
  void reduce_add(T v, memory_order o) const noexcept;
};
```

We also introduce `compare_store`, which is the Atomic Reduction Operation analogous of `compare_exchange`:

```
template <class T>
struct atomic_ref {
  bool compare_exchange_weak(T& expected, T desired, memory_order o) const n
  void compare_store        (T  expected, T desired, memory_order o) const n
};
```

The `reduce_{OP}` and `compare_store` APIs are vectorization safe if the atomic `is_always_lock_free` is true, i.e., they are allowed in Element Access Functions ([algorithms.paralle.defns] (https://eel.is/c++draft/algorithms.parallel.defns#def:element_access_functions)) of parallel algorithms:

```
for_each(par_unseq, ..., [&](auto old, ...) {
    assert(atom.is_lockfree());// Only for lock-free atomics
    atom.store(42);            // OK: vectorization-safe
    atom.reduce_add(1);        // OK: vectorization-safe
    atom.compare_store(1, 2);  // OK: vectorization-safe
    atom.fetch_add(1);         // UB: vectorization-unsafe
    atom.exchange(42);         // UB: vectorization-unsafe
    atom.compare_exchange_weak(old, 42);   // UB: vectorization-unsafe
    atom.compare_exchange_strong(old, 42); // UB: vectorization-unsafe
    while (atom.load() < 42);  // UB: vectorization-unsafe
});
```

Furthermore, we specify non-associative floating-point atomic reduction operations to enable tree-reduction implementations that improve complexity from `O(N)` to `O(log(N))` by minimally increasing the non-determinism which is already inherent to atomic operations. This allows producing the `x == a + (b + c)` outcome in the following example, which enables the optimization that merges the two `reduce_add` operations into a single one:

```
atomic<float> x = a;
x.reduce_add(b, memory_order_relaxed);
x.reduce_add(c, memory_order_relaxed);
// Sound to merge these two operations into one:
// x.store_add(b + c);
```

Applications that need the sequential semantics can use `fetch_add` instead.

Implementation impact

It is correct to conservatively implement `reduce_{OP}` as a call to `fetch_{OP}`. We evaluated the following implementations of unsequenced execution policies, which are not impacted:

- OpenMP simd (https://www.openmp.org/spec-html/5.0/openmpsu42.html) pragma for `unseq` and `par_unseq`, since OpenMP supports atomics within `simd` regions.
- pragma clang loop (https://clang.llvm.org/docs/LanguageExtensions.html#extensions-for-loop-hint-optimizations) is a hint.

Design Alternatives

Enable Atomic Reductions as `fetch_<key>` optimizations

Attempting to improve application performance by implementing compiler-optimizations to leverage Atomic Reduction Operations from `fetch_<key>` APIs whose result is unused has become a rite of passage for compiler engineers, e.g., GCC#509632

(https://gcc.gnu.org/pipermail/gcc-patches/2018-October/509632.html), LLVM#68428 (https://github.com/llvm/llvm-project/issues/68428), LLVM#72747 (https://github.com/llvm/llvm-project/pull/72747), … Unfortunately, "simple" optimization strategies break backward compatibility in the following litmus tests (among others).

Litmus Test 0: from Will Deacon (https://gcc.gnu.org/pipermail/gcc-patches/2018-October/509632.html). Performing the optimization to replace the introduces the `y == 2 && r0 == 1 && r1 == 0` outcome:

```
void thread0(atomic_int* y,atomic_int* x) {
  atomic_store_explicit(x,1,memory_order_relaxed);
  atomic_thread_fence(memory_order_release);
  atomic_store_explicit(y,1,memory_order_relaxed);
}

void thread1(atomic_int* y,atomic_int* x) {
  atomic_fetch_add_explicit(y,1,memory_order_relaxed);
  atomic_thread_fence(memory_order_acquire);
  int r0 = atomic_load_explicit(x,memory_order_relaxed);
}

void thread2(atomic_int* y) {
  int r1 = atomic_load_explicit(y,memory_order_relaxed);
}
```

Litmus Test 1: from Luke Geeson (https://github.com/llvm/llvm-project/issues/68428#issue-1930595855). Performing the optimization of replacing the exchange with a store introduces the `r0 == 0 && y == 2` outcome:

```
void thread0(atomic_int* y,atomic_int* x) {
  atomic_store_explicit(x,1,memory_order_relaxed);
  atomic_thread_fence(memory_order_release);
  atomic_store_explicit(y,1,memory_order_relaxed);
}
void thread1(atomic_int* y,atomic_int* x) {
  atomic_exchange_explicit(y,2,memory_order_release);
  atomic_thread_fence(memory_order_acquire);
  int r0 = atomic_load_explicit(x,memory_order_relaxed);
}
```

In some architectures, Atomic Reduction Operations can write to memory pages or memory locations that are not readable, e.g., MMIO registers on NVIDIA GPUs, and need a reliable programming model that does not depend on compiler-optimizations for functionality.

Naming

We considered the following alternative names:

- `atomic.add` : breaks for logical operations (e.g. `atomic.and` , etc.).
- `atomic.store_add` : clearly states that this operation is a "store" (and not a "load").
- `atomic.reduce_add` : clearly state what this operation is for (reductions).

We considered providing separate version of non-associative floating-point atomic reduction operations with and without the support for tree-reductions, e.g., `atomic.reduce_add` (no tree-reduction support) and `atomic.reduce_add_generalized` (tree-reduction support), but decided against it because atomic operations are inherently non-deterministic, this relaxation only minimally impacts that, and `fetch_add` already provides (and has to continue to provide) the sequential semantics without this relaxation.

Memory Ordering

We choose to support `memory_order_relaxed` , `memory_order_release` , and `memory_order_seq_cst` .

We may need a note stating that replacing the operations in a reduction sequence is only valid as long as the replacement maintains the other ordering properties of the operations as defined in [intro.races]. Examples:

Litmus test 2:

```
// T0:
M1.reduce_add(1, relaxed);
M2.store(1, release);
M1.reduce_add(1, relaxed);

// T1:
r0 = M2.load(acquire);
r1 = M1.load(relaxed);

// r0 == 0 || r1 >= 1
```

Litmus test 3:

```
// T0
M1.reduce_add(1, seq_cst);
M2.reduce_add(1, seq_cst);
M1.reduce_add(1, seq_cst);

// T1
r0 = M2.load(seq_cst);
r1 = M1.load(seq_cst);

// r0 == 0 || r1 >= 1
```

Unresolved question: Are tree reductions only supported for `memory_order_relaxed` ? No, see litmus tests for release and seq_cst.

Formalization

Herd already support these for `STADD` on Arm, and the NVIDIA Volta Memory Model supports these for `red` and `multimem.red` on PTX. If we decide to pursue this exposure direction, this proposal would benefit from extending Herd's RC11 with reduction sequences for floating-point.

Wording

▶ Do NOT modify [intro.execution.10]!

Add to [intro.races] (https://eel.is/c++draft/intro.races) before [intro.races.14] (https://eel.is/c++draft/intro.races#14):

Review Note: the intent of this wording change is to enable tree-reduction implementations for non-associative floating-point atomic reduction operations which, in this proposal, are only `reduce_add` and `reduce_sub` . SG1 agreed that the single floating point reduction operations should allow "fast math" (allow tree reductions) because programs can get precise results using `fetch_add` / `sub` . Alternative wording strategies using `GENERALIZED_NONCOMMUTATIVE_SUM` were explored but are not viable since only two values are involved.
Review Note: Since other proposals in flight add `fetch_mul` and `fetch_div` , and those would need to extend this, this revision of this paper shows how that would look like by introducing "multiplicative reduction sequences".
Review Note: For integers, implementations are already allowed to perform tree reductions, as long as they don't violate any other ordering rules. The only goal of the following reduction sequence wording is to allow the exact same thing for floats, since implementation cannot perform tree reductions because the ops are not associative and impact the results. Maybe

instead of adding all of this wording, we could instead say that reduction operations on floats may be re-ordered or re-associated "as if" they were operations on integers? For example, we could add a Remark to the floating-point specialization of atomic and atomic_ref that just says: "Remark: The associativity of floating-point atomic reduction operations is the same as that of integers."

Option A.0: this option specifies the optimization that is allowed for floating-point atomic reduction operations..

1. A *reduction sequence* is a maximal contiguous sub-sequence of side effects in the modification order of M, where each operation is an atomic reduction operation. If replacing two adjacent integer operations in a reduction sequence is not observable, this same replacement may be performed for two adjacent floating-point operations in a reduction sequence.
   [Note 1: Combining adjacent operations in a reduction sequence is often not observable for integer operations but is often observable for floating-point operations due to associativity. This paragraph enables implementations to optimize floating-point atomic reduction operations using tree reductions. - end note]

Alternative A.1: Replace the last sentence in the paragraph with: "Any two adjacent floating-point operations in a reduction sequence can be replaced by a single operation as if the operations were integer operations.".

Option B: this option explicitly specify the allowed replacements for floating-point operations using a table.

1. A *reduction sequence* is a maximal contiguous sub-sequence of side effects in the modification order of M, where each operation is an atomic reduction operation. Any two adjacent floating-point operations in a reduction sequence whose replacement does not contradict any memory ordering requirements can be replaced by a single operation, recursively, as specified in Table [tab.red.seq.replacement]. The replaced operation uses the largest memory order of both operations according to `memory_order_relaxed` < `memory_order_release` < `memory_order_seq_cst`. For signed integer types, the intermediate computation is performed as if the objects `a` and `b` were converted to their corresponding unsigned types, and its result converted back to the signed type.
   [Note 1: Replacing any two adjacent operations in a reduction sequence is only valid if the replacement maintains the other ordering properties of the operations as defined in [intro.races]. The assertion in the following example holds:

```
atomic<int> M0 = 0, M1 = 0;
void thread0() {
  M0.reduce_add(1, memory_order_relaxed);
  M1.store(1, memory_order_release);
  M0.reduce_add(1, memory_order_relaxed);
}
void thread1() {
  auto r0 = M1.load(memory_order_acquire);
  auto r1 = M0.load(memory_order_relaxed);
  assert(r0 == 0 || r1 >= 1);
}
```

That is, replacing the two adjacent `M0.reduce_add` on `thread0` with a single `reduce_add` after the `M1` release store as follows:

```
void thread0() {
  M1.store(1, memory_order_release);
  M0.reduce_add(2, memory_order_relaxed);
}
```

is not allowed, but replacing them before the store as follows:

- end note]

Table [tab.red.seq.replacement]: Allowed replacements of two adjacent floating-point atomic reduction operations `reduce <key>` within a floating-point atomic reduction sequence on memory location M, where the value parameter of the first atomic operation is `a`, and that of the second is `b`.

Second Op First Op	**add**	**sub**	**min**	**max**
`add`	`add(a + b)`	`add(a − b)`	n/a	n/a
`sub`	`add(b − a)`	`sub(a + b)`	n/a	n/a
`min`	n/a	n/a	`min(min(a,b))`	n/a
`max`	n/a	n/a	n/a	`max(max(a,b))`

Option B.1: this option may be extended to mul and div operations as follows:

Second Op First Op	**mul**	**div**
mul	mul(a * b)	mul(a / b)
div	mul(b / a)	div(a * b)

Option B.2: this option explicitly specify the allowed replacements for floating-point operations without a table.

1. … as specified below, recursively:
   - `M.reduce_add(a, o0)` and `M.reduce_add(b, o1)` can be replaced with `M.reduce_add(a + b, max_order(o0, o1))`,
   - `M.reduce_sub(a, o0)` and `M.reduce_sub(b, o1)` can be replaced with `M.reduce_sub(a + b, max_order(o0, o1))`,
   - `M.reduce_add(a, o0)` and `M.reduce_sub(b, o1)` can be replaced with `M.reduce_add(a - b, max_order(o0, o1))`,
   - `M.reduce_sub(a, o0)` and `M.reduce_add(b, o1)` can be replaced with `M.reduce_add(b - a, max_order(o0, o1))`,
   - `M.reduce_mul(a, o0)` and `M.reduce_mul(b, o1)` can be replaced with `M.reduce_mul(a * b, max_order(o0, o1))`,
   - `M.reduce_div(a, o0)` and `M.reduce_div(b, o1)` can be replaced with `M.reduce_div(a * b, max_order(o0, o1))`,
   - `M.reduce_mul(a, o0)` and `M.reduce_div(b, o1)` can be replaced with `M.reduce_mul(a / b, max_order(o0, o1))`,
   - `M.reduce_div(a, o0)` and `M.reduce_mul(b, o1)` can be replaced with `M.reduce_mul(b / a, max_order(o0, o1))`,
   - `M.reduce_min(a, o0)` and `M.reduce_min(b, o1)` can be replaced with `M.reduce_min(min(a, b), max_order(o0, o1))`,
   - `M.reduce_max(a, o0)` and `M.reduce_max(b, o1)` can be replaced with `M.reduce_max(max(a, b), max_order(o0, o1))`.

Add to [algorithms.parallel.defns] (https://eel.is/c++draft/algorithms.parallel.defns#5):

Review note: the first bullet says which standard library functions are, in general, vectorization unsafe, and the second bullet carves out exceptions.

Review note: the current wording intent is for `fetch_add(relaxed)` to be vectorization-unsafe, but the standard words in [intro.execution.10] (see above) that were supposed to fix that, do not. So we currently ban those here as a drive-by change that should be filled as LWG defect report, but that this proposal would then need to carve out an exception for atomic

reduction operations.
Review Note: SG1 requested making "relaxed", "release", and "seq_cst" atomic reduction operations not be vectorization unsafe, and that is the intent of the words below.

> A standard library function is vectorization-unsafe if:
>
> - it is specified to synchronize with another function invocation, or another function invocation is specified to synchronize with it,~~and~~or it is a relaxed atomic operation [atomics.order] (https://eel.is/c++draft/atomics.order), and
> - ~~if~~ it is not a memory allocation or deallocation function or lock-free atomic reduction operation.
>
> [Note 2: Implementations must ensure that internal synchronization inside standard library functions does not prevent forward progress when those functions are executed by threads of execution with weakly parallel forward progress guarantees. — end note]
>
> [Example 2:
>
> ```
> int x = 0;
> std::mutex m;
> void f() {
>   int a[] = {1,2};
>   std::for_each(std::execution::par_unseq, std::begin(a), std::end(a), [&](in
>     std::lock_guard<mutex> guard(m); // incorrect: lock_guard constructor cal
>     ++x;
>   });
> }
> ```
>
> The above program may result in two consecutive calls to m.lock() on the same thread of execution (which may deadlock), because the applications of the function object are not guaranteed to run on different threads of execution. — end example]

Forward progress

Modify [intro.progress#1] (https://eel.is/c++draft/intro.progress#1) as follows:

Review Note: SG1 design intent is that Reduction operations are not a step, but infinite loops around reductions operations are not UB (practically implementations can assume reductions are finite). Gonzalo's note: such loops cause all threads to loose forward progress.
Review Note: this change makes Atomic Reduction Operations and Atomic Stores/Fences asymmetric, but making this change for Atomic Stores/Fences is a breaking change to be

pursued in a separate paper per SG1 feedback. When adding these new operations, it does make sense to make this change, to avoid introducing undesired semantics that would be a breaking change to fix.

> The implementation may assume that any thread will eventually do one of the following:
>
> 1. terminate,
> 2. invoke the function std::this_thread::yield ([thread.thread.this]),
> 3. make a call to a library I/O function,
> 4. perform an access through a volatile glvalue, or
> 5. perform a synchronization operation or an atomic operation other than an atomic reduction operation [atomics.order] (https://eel.is/c++draft/atomics.order), or
> 6. continue execution of a trivial infinite loop ([stmt.iter.general]).
>
> [Note 1: This is intended to allow compiler transformations such as removal of empty loops, even when termination cannot be proven. — end note]

Modify [intro.progress#3] (https://eel.is/c++draft/intro.progress#3) as follows:

Review note: same note as above about exempting atomic stores in a separate paper.

> During the execution of a thread of execution, each of the following is termed an execution step:
>
> 1. termination of the thread of execution,
> 2. performing an access through a volatile glvalue, or
> 3. completion of a call to a library I/O function, or a synchronization operation~~,~~ or an atomic operation other than an atomic reduction operation [atomics.order] (https://eel.is/c++draft/atomics.order).

No acquire sequences support

Modify [atomics.fences] (https://eel.is/c++draft/atomics.fences) as follows:

33.5.11 Fences[atomics.fences]

1. This subclause introduces synchronization primitives called fences. Fences can have acquire semantics, release semantics, or both. A fence with acquire semantics is called an acquire fence. A fence with release semantics is called a release fence.
2. A release fence A synchronizes with an acquire fence B if there exist atomic operations X and Y, where Y is not an atomic reduction operation [atomics.order] (https://eel.is/c++draft/atomics.order), both operating on some atomic object M, such that A is sequenced before X, X modifies M, Y is sequenced before B, and Y reads the value written by X or a value written by any side effect in the hypothetical release sequence X would head if it were a release operation.
3. A release fence A synchronizes with an atomic operation B that performs an acquire operation on an atomic object M if there exists an atomic operation X such that A is sequenced before X, X modifies M, and B reads the value written by X or a value written by any side effect in the hypothetical release sequence X would head if it were a release operation.
4. An atomic operation A that is a release operation on an atomic object M synchronizes with an acquire fence B if there exists some atomic operation X on M such that X is sequenced before B and reads the value written by A or a value written by any side effect in the release sequence headed by A.

Atomic Reduction Operation APIs

The following operations perform arithmetic computations. The correspondence among key, operator, and computation is specified in Table 145 (https://eel.is/c++draft/atomics#tab:atomic.types.int.comp):

key	op	computation
add	+	addition
sub	-	subtraction
max		maximum
min		minimum
and	&	bitwise and
or	|	bitwise inclusive or
xor	^	bitwise exclusive or

Add to [atomics.syn] (https://eel.is/c++draft/atomics.syn):

```
namespace std {
// [atomics.nonmembers], non-member functions
...
template<class T>
 void atomic_compare_store(atomic<T>*,                // freestanding
                           typename atomic<T>::value_type*,
                           typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_compare_store(volatile atomic<T>*,       // freestanding
                           typename atomic<T>::value_type*,
                           typename atomic<T>::value_type) noexcept;

template<class T>
 void atomic_reduce_add(volatile atomic<T>*,          // freestanding
                        typename atomic<T>::difference_type) noexcept;
template<class T>
 void atomic_reduce_add(atomic<T>*,                   // freestanding
                        typename atomic<T>::difference_type) noexcept;
template<class T>
 void atomic_reduce_add_explicit(volatile atomic<T>*,// freestanding
                                 typename atomic<T>::difference_type,
                                 memory_order) noexcept;
template<class T>
 void atomic_reduce_add_explicit(atomic<T>*,          // freestanding
                                 typename atomic<T>::difference_type,
                                 memory_order) noexcept;
template<class T>
 void atomic_reduce_sub(volatile atomic<T>*,          // freestanding
                        typename atomic<T>::difference_type) noexcept;
template<class T>
 void atomic_reduce_sub(atomic<T>*,                   // freestanding
                        typename atomic<T>::difference_type) noexcept;
template<class T>
 void atomic_reduce_sub_explicit(volatile atomic<T>*,// freestanding
                                 typename atomic<T>::difference_type,
                                 memory_order) noexcept;
template<class T>
 void atomic_reduce_sub_explicit(atomic<T>*,          // freestanding
                                 typename atomic<T>::difference_type,
                                 memory_order) noexcept;
template<class T>
 void atomic_reduce_and(volatile atomic<T>*,          // freestanding
                        typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_reduce_and(atomic<T>*,                   // freestanding
                        typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_reduce_and_explicit(volatile atomic<T>*,// freestanding
                                 typename atomic<T>::value_type,
```

```
                                    memory_order) noexcept;
template<class T>
 void atomic_reduce_and_explicit(atomic<T>*,          // freestanding
                                    typename atomic<T>::value_type,
                                    memory_order) noexcept;
template<class T>
 void atomic_reduce_or(volatile atomic<T>*,           // freestanding
                       typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_reduce_or(atomic<T>*,                    // freestanding
                       typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_reduce_or_explicit(volatile atomic<T>*, // freestanding
                                   typename atomic<T>::value_type,
                                   memory_order) noexcept;
template<class T>
 void atomic_reduce_or_explicit(atomic<T>*,           // freestanding
                                    typename atomic<T>::value_type,
                                    memory_order) noexcept;
template<class T>
 void atomic_reduce_xor(volatile atomic<T>*,          // freestanding
                        typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_reduce_xor(atomic<T>*,                   // freestanding
                        typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_reduce_xor_explicit(volatile atomic<T>*,// freestanding
                                    typename atomic<T>::value_type,
                                    memory_order) noexcept;
template<class T>
 void atomic_reduce_xor_explicit(atomic<T>*,          // freestanding
                                    typename atomic<T>::value_type,
                                    memory_order) noexcept;
template<class T>
 void atomic_reduce_max(volatile atomic<T>*,          // freestanding
                        typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_reduce_max(atomic<T>*,                   // freestanding
                        typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_reduce_max_explicit(volatile atomic<T>*, // freestanding
                                    typename atomic<T>::value_type,
                                    memory_order) noexcept;
template<class T>
 void atomic_reduce_max_explicit(atomic<T>*,          // freestanding
                                    typename atomic<T>::value_type,
                                    memory_order) noexcept;
template<class T>
 void atomic_reduce_min(volatile atomic<T>*,          // freestanding
```

```
                          typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_reduce_min(atomic<T>*,                    // freestanding
                          typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_reduce_min_explicit(volatile atomic<T>*, // freestanding
                                 typename atomic<T>::value_type,
                                 memory_order) noexcept;
template<class T>
 void atomic_reduce_min_explicit(atomic<T>*,          // freestanding
                                 typename atomic<T>::value_type,
                                 memory_order) noexcept;
}
```

Add to [atomics.ref.generic] (https://eel.is/c++draft/atomics.ref.generic):

```
namespace std {
  template<class T> struct atomic_ref {
    ...
  public:
    ...
    bool compare_exchange_strong(T&, T,
                                 memory_order = memory_order::seq_cst) const noexc

    void compare_store(T, T, memory_order = memory_order::seq_cst) const noexcept;
    // ...
  };
}
```

Add to [atomics.ref.ops] (https://eel.is/c++draft/atomics.ref.ops):

```
void compare_store(T expected, T desired,
                   memory_order order = memory_order::seq_cst) const noexcept;
```

- *Preconditions*: `order` is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.
- *Effects*: Atomically compares the value representation of the value referenced by `*ptr` for equality with the value in `expected`, and if `true`, replaces the value pointed to by `*ptr` with that in `desired`. If and only if the comparison is `true`, memory is affected according to the value of `order`; otherwise, memory is not affected. This operation is an atomic read-modify-write operations ([intro.races]) and an atomic reduction operations on the value referenced by `*ptr`.

Add to [atomics.ref.int] (https://eel.is/c++draft/atomics.ref.int):

```
namespace std {
  template<> struct atomic_ref<integral-type> {
    ...
   public:
    ...
    void compare_store(integral-type expected, integral-type desired,
                       memory_order order = memory_order::seq_cst) const noexcept;
    void reduce_add(integral-type,
                       memory_order = memory_order::seq_cst) const noexcept;
    void reduce_sub(integral-type,
                    memory_order = memory_order::seq_cst) const noexcept;
    void reduce_and(integral-type,
                       memory_order = memory_order::seq_cst) const noexcept;
    void reduce_or(integral-type,
                    memory_order = memory_order::seq_cst) const noexcept;
    void reduce_xor(integral-type,
                       memory_order = memory_order::seq_cst) const noexcept;
    void reduce_max(integral-type,
                       memory_order = memory_order::seq_cst) const noexcept;
    void reduce_min(integral-type,
                       memory_order = memory_order::seq_cst) const noexcept;

}


void reduce_<key>(integral-type operand,
                  memory_order order = memory_order_seq_cst) const noexcept;
```

- *Preconditions*: `order` is `memory_order_relaxed`, `memory_order_release`, or `memory_order_seq_cst`.
- *Effects*: Atomically replaces the value referenced by `*ptr` with the result of the computation applied to the value referenced by `*ptr` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations ([intro.races]) **and atomic reduction operations**.
- *Remarks*: Except for `reduce_max` and `reduce_min`, for signed integer types, the result is as if the object value and parameters were converted to their corresponding unsigned types, the computation performed on those types, and the result converted back to the signed type.
  [Note 2: There are no undefined results arising from the computation. — end note]
  [Note 3: A lock-free atomic reduction operation is not vectorization-unsafe ([algorithms.parallel.defns]). — end note]

For `reduce_max` and `reduce_min`, the maximum and minimum computation is performed as if by `max` and `min` algorithms ([alg.min.max]), respectively, with the object value and the first parameter as the arguments.

Add to [atomics.ref.float] ():

```
namespace std {
  template<> struct atomic_ref<floating-point-type> {
    ...
  public:
    ...
    void compare_store(floating-point-type expected, floating-point-type desired,
                       memory_order = memory_order::seq_cst) const noexcept;
    void reduce_add(floating-point-type,
                    memory_order = memory_order::seq_cst) const noexcept;
    void reduce_sub(floating-point-type,
                    memory_order = memory_order::seq_cst) const noexcept;
  };
}
```

Review note: add reduce_minimum, _minimum_num, _maximum, and _maximum_num once 3008 is merged.

```
void reduce_key(floating-point-type operand,
                memory_order order = memory_order::seq_cst) const noexcept;
```

- *Preconditions*: `order` is `memory_order_relaxed`, `memory_order_release`, or `memory_order_seq_cst`.
- *Effects*: Atomically replaces the value referenced by `*ptr` with the result of the computation applied to the value referenced by `*ptr` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations ([intro.races]) **and atomic reduction operations**.
- *Remarks*: If the result is not a representable value for its type ([expr.pre]), the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on floating-point-type should conform to the `std::numeric_limits<floating-point-type>` traits associated with the floating-point type ([limits.syn]). The floating-point environment ([cfenv]) for atomic arithmetic operations on floating-point-type may be different than the calling thread's floating-point environment.
- [Note 3: A lock-free atomic reduction operation is not vectorization-unsafe ([algorithms.parallel.defns]). — end note]

Add to [atomics.ref.pointer] (https://eel.is/c++draft/atomics.ref.generic#atomics.ref.pointer):

```
namespace std {
 template<class T> struct atomic_ref<T*> {
    ...
  public:
    ...
    void compare_store(T* expected, T* desired,
                       memory_order = memory_order::seq_cst) const noexcept;

    void reduce_add(difference_type,
                    memory_order = memory_order::seq_cst) const noexcept;
    void reduce_sub(difference_type,
                    memory_order = memory_order::seq_cst) const noexcept;
    void reduce_max(T*,
                    memory_order = memory_order::seq_cst) const noexcept;
    void reduce_min(T*,
                    memory_order = memory_order::seq_cst) const noexcept;
  };
}



void reduce_key(difference_type operand,
                memory_order order = memory_order::seq_cst) const noexcept;
```

- *Mandates*: `T` is a complete object type.ins>
- *Preconditions*: `order` is `memory_order_relaxed`, `memory_order_release`, or `memory_order_seq_cst`.
- *Effects*: Atomically replaces the value referenced by `*ptr` with the result of the computation applied to the value referenced by `*ptr` and the given operand. Memory is affected according to the value of order. These operations are atomic read-modify-write operations ([intro.races]) **and atomic reduction operations**.
- *Remarks*: The result may be an undefined address, but the operations otherwise have no undefined behavior.
  For `fetch_max` and `fetch_min`, the maximum and minimum computation is performed as if by `max` and `min` algorithms ([alg.min.max]), respectively, with the object value and the first parameter as the arguments.
  [Note 1: If the pointers point to different complete objects (or subobjects thereof), the `<` operator does not establish a strict weak ordering (Table 29, [expr.rel]). — end note]
  [Note 2: A lock-free atomic reduction operation is not vectorization-unsafe ([algorithms.parallel.defns]). — end note]

Add to [atomics.types.generic.general] (https://eel.is/c++draft/atomics.types.generic.general):

```
namespace std {
  template<class T> struct atomic {

    void compare_store(T, T, memory_order) volatile noexcept;
    void compare_store(T, T, memory_order) noexcept;

  };
}
```

Add to [atomics.types.operations] (https://eel.is/c++draft/atomics.types.operations):

```
void compare_store(T expected, T desired,
                   memory_order order = memory_order::seq_cst) const noexcept;
```

- *Preconditions*: `order` is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.
- *Effects*: Atomically compares the value representation of the value pointed to by `this` for equality with that of `expected`, and if `true`, replaces the value pointed to by `this` with that in `desired`. If and only if the comparison is `true`, memory is affected according to the value of `order`; otherwise, memory is not affected. This operation is an atomic read-modify-write operations ([intro.races]) and an atomic reduction operations on the memory pointed to by `this`.

Add to [atomics.types.int] (https://eel.is/c++draft/atomics.types.int):

```
namespace std {
  template<> struct atomic<integral-type> {
    ...

    void compare_store(integral-type, integral-type,
                       memory_order,) volatile noexcept;
    void compare_store(integral-type, integral-type,
                       memory_order) noexcept;

    void reduce_add(integral-type,
                    memory_order = memory_order::seq_cst) volatile noexcept;
    void reduce_add(integral-type,
                    memory_order = memory_order::seq_cst) noexcept;
    void reduce_sub(integral-type,
                    memory_order = memory_order::seq_cst) volatile noexcept;
    void reduce_sub(integral-type,
                    memory_order = memory_order::seq_cst) noexcept;
    void reduce_and(integral-type,
                    memory_order = memory_order::seq_cst) volatile noexcept;
    void reduce_and(integral-type,
                    memory_order = memory_order::seq_cst) noexcept;
    void reduce_or(integral-type,
                    memory_order = memory_order::seq_cst) volatile noexcept;
    void reduce_or(integral-type,
                    memory_order = memory_order::seq_cst) noexcept;
    void reduce_xor(integral-type,
                    memory_order = memory_order::seq_cst) volatile noexcept;
    void reduce_xor(integral-type,
                    memory_order = memory_order::seq_cst) noexcept;
    void reduce_max(integral-type,
                    memory_order = memory_order::seq_cst) volatile noexcept;
    void reduce_max(integral-type,
                    memory_order = memory_order::seq_cst) noexcept;
    void reduce_min(integral-type,
                    memory_order = memory_order::seq_cst) volatile noexcept;
    void reduce_min(integral-type,
                    memory_order = memory_order::seq_cst) noexcept;

  };
}



void reduce_<key>(integral-type operand,
          memory_order order = memory_order_seq_cst) volatile noexcept;
void reduce_<key>(integral-type operand,
          memory_order order = memory_order_seq_cst) noexcept;
```

- *Constraints*: For the `volatile` overload of this function, `is_always_lock_free` is true.
- *Preconditions*: `order` is `memory_order_relaxed`, `memory_order_release`, or `memory_order_seq_cst`.
- *Effects*: Atomically replaces the value pointed to by `this` with the result of the computation applied to the value pointed to by `this` and the given operand. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations ([intro.multithread]) **and atomic reduction operations**.
- *Remarks*: Except for `reduce_max` and `reduce_min`, for signed integer types the result is as if the object value and parameters were converted to their corresponding unsigned types, the computation performed on those types, and the result converted back to the signed type.
  [Note 2: There are no undefined results arising from the computation. — end note]
- [Note 3: A lock-free atomic reduction operation is not vectorization-unsafe ([algorithms.parallel.defns]). — end note]
  For `reduce_max` and `reduce_min`, the maximum and minimum computation is performed as if by `max` and `min` algorithms ([alg.min.max]), respectively, with the object value and the first parameter as the arguments.

Add to [atomics.types.float] (https://eel.is/c++draft/atomics.types.float):

```
namespace std {
  template<> struct atomic<floating-point-type> {
    ...

    void compare_store(floating-point-type, floating-point-type,
                       memory_order, memory_order) volatile noexcept;
    void compare_store(floating-point-type&, floating-point-type,
                       memory_order, memory_order) noexcept;

    void reduce_add(floating-point-type,
                    memory_order = memory_order::seq_cst) volatile noexcept;
    void reduce_add(floating-point-type,
                    memory_order = memory_order::seq_cst) noexcept;
    void reduce_sub(floating-point-type,
                    memory_order = memory_order::seq_cst) volatile noexcept;
    void reduce_sub(floating-point-type,
                    memory_order = memory_order::seq_cst) noexcept;

  };
}
```

Review note: add reduce_minimum, _minimum_num, _maximum, and _maximum_num once 3008 is merged.

```
void reduce_key(T operand,
      memory_order order = memory_order::seq_cst) volatile noexcept;
void reduce_key(T operand,
      memory_order order = memory_order::seq_cst) noexcept;
```

- *Constraints*: For the `volatile` overload of this function, `is_always_lock_free` is true.
- *Preconditions*: `order` is `memory_order_relaxed`, `memory_order_release`, or `memory_order_seq_cst`.
- *Effects*: Atomically replaces the value pointed to by `this` with the result of the computation applied to the value pointed to by `this` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations ([intro.multithread]) **and atomic reduction operations**.
- *Remarks*: If the result is not a representable value for its type ([expr.pre]) the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on `floating-point-type` should conform to the `std::numeric_limits<floating-point-type>` traits associated with the floating-point type ([limits.syn]). The floating-point environment ([cfenv]) for atomic arithmetic operations on `floating-point-type` may be different than the calling thread's floating-point environment.

Add to [atomics.types.pointer] (https://eel.is/c++draft/atomics.types.pointer):

```
namespace std {
  template<class T> struct atomic<T*> {
    ...

    void compare_store(T*, T*,
            memory_order = memory_order::seq_cst) volatile noexcept;
    void compare_store(T*, T*,
            memory_order = memory_order::seq_cst) noexcept;

    void reduce_add(ptrdiff_t,
      memory_order = memory_order::seq_cst) volatile noexcept;
    void reduce_add(ptrdiff_t,
      memory_order = memory_order::seq_cst) noexcept;
    void reduce_sub(ptrdiff_t,
      memory_order = memory_order::seq_cst) volatile noexcept;
    void reduce_sub(ptrdiff_t,
      memory_order = memory_order::seq_cst) noexcept;
    void reduce_max(T*,
      memory_order = memory_order::seq_cst) volatile noexcept;
    void reduce_max(T*,
      memory_order = memory_order::seq_cst) noexcept;
    void reduce_min(T*,
      memory_order = memory_order::seq_cst) volatile noexcept;
    void reduce_min(T*,
      memory_order = memory_order::seq_cst) noexcept;
  };
}
```

```
void reduce_key(ptrdiff_t operand,
     memory_order order = memory_order::seq_cst) volatile noexcept;
void reduce_key(ptrdiff_t operand,
     memory_order order = memory_order::seq_cst) noexcept;
```

- *Constraints*: For the `volatile` overload of this function, `is_always_lock_free` is true.
- *Mandates*: `T` is a complete object type.
  [Note 1: Pointer arithmetic on void* or function pointers is ill-formed. — end note]
- *Effects*: Atomically replaces the value pointed to by this with the result of the computation applied to the value pointed to by `this` and the given operand. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations ([intro.multithread]).
- *Remarks*: The result may be an undefined address, but the operations otherwise have no undefined behavior.
  For `reduce_max` and `reduce_min`, the maximum and minimum computation is performed

as if by `max` and `min` algorithms ([alg.min.max]), respectively, with the object value and the first parameter as the arguments.

[Note 2: If the pointers point to different complete objects (or subobjects thereof), the `<` operator does not establish a strict weak ordering (Table 29, [expr.rel]). — end note]

Add to [util.smartptr.atomic.shared] (https://eel.is/c++draft/util.smartptr.atomic.shared):

```
namespace std {
  template<class T> struct atomic<shared_ptr<T>> {
    ...

    void compare_store(shared_ptr<T> expected, shared_ptr<T> desired,
      memory_order order = memory_order::seq_cst) noexcept;

  };
}
```

Add to [util.smartptr.atomic.weak] (https://eel.is/c++draft/util.smartptr.atomic.weak):

```
namespace std {
  template<class T> struct atomic<weak_ptr<T>> {
    ...

    void compare_store(weak_ptr<T> expected, weak_ptr<T> desired,
      memory_order order = memory_order::seq_cst) noexcept;

  };
}
```